

Information Network Security Administration

INSA Cyber Talent Summer Camp 5th Batch, Jimma

Graduation Project Report
Project Title:- CHEM-SHIELD

Group Members:

1. Hasenat Abdu …… CTC-723-26
2. Wudase Kenu …… CTC-1906-26
3. Biruktawit Sebsibe …… CTC-6596-26
4. Firomsa Abdisa …… CTC-1602-26

Submission to: Mr. Biniyam Bahiru
Submission Date: October 28, 2026 G.C

Table of Contents

- List of Tables
- List of Abbreviations
- Executive Summary
- Chapter One: Introduction
    - 1.1 Background of the Project
    - 1.2 Problem Statement
    - 1.3 General Objective
    - 1.4 Specific Objectives
    - 1.5 Scope and Limitations
    - 1.6 Significance of the Project
    - 1.7 Report Organization
- Chapter Two: Literature Review / Related Work
    - 2.1 Introduction
    - 2.2 Existing Solutions and Concepts
    - 2.3 Limitations of Existing Solutions
    - 2.4 Project Positioning
    - 2.5 Chapter Summary
- Chapter Three: Methodology and System Design
    - 3.1 Development Methodology and Justification
    - 3.2 Requirements Analysis
    - 3.3 System Architecture
    - 3.4 System Design and Workflow
    - 3.5 Data and Database Design
    - 3.6 Tools, Technologies, and Development Environment
    - 3.7 Chapter Summary
- Chapter Four: Implementation and Results
    - 4.1 Introduction
    - 4.2 Implementation Status Summary
    - 4.3 Implementation Details
    - 4.4 Major Features and Interfaces
    - 4.5 Testing Methods and Results
    - 4.6 Evaluation, Results, and Discussion
    - 4.7 Challenges Encountered and Solutions Adopted
    - 4.8 Chapter Summary
- Chapter Five: Conclusion and Future Work
    - 5.1 Introduction
    - 5.2 Conclusions
    - 5.3 Achievements and Contributions
    - 5.4 Limitations
    - 5.5 Recommendations and Future Work
    - 5.6 Final Remarks
- References
- Appendix A: Repository Layout

- Appendix B: Installation and Execution Guide
- Appendix C: User Guide and Demonstration Procedure
- Appendix D: Configuration Details
- Appendix E: Automated Test Inventory

List of Tables

No.	Title
Table 2.1	Comparison of single-source monitoring and CHEM-SHIELD
Table 3.1	Stakeholders and actors
Table 3.2	Functional requirements
Table 3.3	Non-functional requirements
Table 3.4	Virtual PLC holding-register map
Table 3.5	Normalised event format
Table 3.6	Risk levels and thresholds
Table 3.7	Services and ports
Table 3.8	Technology stack
Table 4.1	Implementation status of each component
Table 4.2	Testing strategy and targets
Table 4.3	Automated test results by area
Table 4.4	Scenario results: normal, network-only, physical-only, correlated
Table 4.5	Physical ML detection results by scenario
Table 4.6	Test cases that remain pending
Table 4.7	Challenges and adopted solutions
Table 5.1	Evaluation of outcomes against objectives
Table 5.2	Suggested roadmap

List of Abbreviations

Abbreviation	Meaning
ABAC	Attribute-Based Access Control
API	Application Programming Interface
CSRF	Cross-Site Request Forgery
FPR / FNR	False Positive Rate / False Negative Rate
HMI	Human–Machine Interface
ICS	Industrial Control System
IDS	Intrusion Detection System
ML	Machine Learning
OT	Operational Technology
PLC	Programmable Logic Controller

Executive Summary

Industrial plants depend on computers that read sensors and drive motors, pumps and valves. These computers, known as operational technology (OT), mostly use protocols such as Modbus TCP that were designed without authentication. A valid-looking command can therefore be harmful, and the network layer alone cannot tell. A monitor that watches only the physical process has the opposite problem: it sees that a machine behaves strangely but not why. This project, CHEM-SHIELD, tests the idea that the two kinds of evidence are more useful together than apart.

CHEM-SHIELD is a defensive research prototype that runs entirely on the loopback interface (127.0.0.1) of one computer. It does not connect to any real industrial, military or SCADA equipment. The system has a virtual Modbus TCP PLC, a physical process simulator, and an operator HMI. A network layer replays Zeek, Suricata, Modbus-monitor and operating-system log fixtures and scores them with an Isolation Forest implemented in the project. A second layer applies scikit-learn's Isolation Forest to simulated telemetry. A correlation engine links network and physical events on the same asset within 10 seconds. A risk engine combines them as `0.40 × network score + 0.60 × physical score`, multiplied by asset criticality, and assigns LOW, MEDIUM, HIGH or CRITICAL. Results are presented in the operator HMI, a role-based Admin console with a hash-chained audit trail, a Command Center for guided simulations, and a standalone OT Defense Dashboard.

The measurements are mixed and are reported as they are. On the current synthetic data the physical model flagged 339 of 7,200 normal readings in a one-hour run (4.71%, or 339 false alarms per hour), detected 87.5% of the stealthy in-range scenario, and detected only 7.5% of slow gradual drift. In the fixture-based chain run, a network anomaly and a physical anomaly on the same asset were joined into one incident scored 0.835 (CRITICAL), while each source alone would have reached only MEDIUM. The automated test suite contains 351 tests; all passed in our verification run. A fused-versus-single-source experiment over many repetitions, live packet capture, and testing against real equipment were not done. These limits are described in Chapters Four and Five.

Chapter One: Introduction

1.1 Background of the Project

Most people connect cybersecurity with data: passwords, files and websites. Many important systems do more than process data. A water treatment plant, a power station, a chemical process and a factory line all move, heat or pressurise something in the physical world. The computers that run them are called operational technology. A sensor measures a quantity such as speed, temperature or pressure. A programmable logic controller (PLC) reads the sensors, applies control logic, and sends commands to actuators such as motors and valves. An operator watches and adjusts the process through a human–machine interface (HMI).

These parts talk to each other over industrial protocols. Modbus TCP is one of the most common. In Modbus, a controller exposes numbered registers and any client that can reach it can read or write

them. The protocol was designed for simple, reliable communication on closed networks and has no login and no encryption [3]. When such equipment is connected to wider networks, a cyber event can become a physical event. A single changed register can make a motor overspeed or a temperature rise.

Security practice for OT therefore differs from office IT. Safety and availability come first, equipment can stay in service for decades, and patching is difficult because stopping a process is costly [1]. Two kinds of monitoring are commonly used. Network monitoring tools such as Zeek and Suricata record who talked to whom [7], [8]. Physical monitoring checks whether the machine is behaving normally. Each answers a different question, and neither alone tells the operator whether a suspicious message actually produced an abnormal physical result.

CHEM-SHIELD was built to study this gap in a safe setting. Real OT equipment is expensive, risky to experiment on, and usually off-limits. A software laboratory lets students repeat experiments and study detection ideas without any real infrastructure. The project does not model any specific facility. It uses a generic simulated motor and process, because the research question concerns detection and evidence correlation and not the details of a particular production line.

1.2 Problem Statement

Network-security monitoring can detect suspicious communication, but it cannot tell whether a protocol-valid command produced an abnormal physical state. A write of a new target speed to the PLC looks the same whether the real operator or an unauthorised sender made it. Physical anomaly detection can notice that speed, temperature or pressure has left the learned normal pattern, but an unusual reading has many possible causes, such as wear, a failing sensor or a maintenance task. It gives no security context and, if it is the only alarm source, it may produce many alerts that the operator cannot rank.

The specific problems this project addresses are:

- **Single-source blind spots.** A network monitor does not see physical consequences, and a physical monitor does not see the communication that may have caused them.
- **Hidden slow changes.** A model that scores each reading against the normal range can miss slow drift and in-range manipulation, which are exactly the cases where other evidence is needed.
- **Unranked alerts.** Without a way to combine and rank evidence, operators receive isolated alerts instead of one explained priority.
- **Limited access to test environments.** Students and researchers rarely have a safe, repeatable OT laboratory in which to test detection ideas.

The research problem is to determine whether combining network evidence and physical-process evidence improves the identification and prioritisation of suspicious cyber-physical events in a controlled OT simulation.

1.3 General Objective

The general objective of the project is to design and build a lightweight, AI-assisted early-warning framework that detects suspicious network activity and abnormal process behaviour in an isolated simulated OT laboratory, correlates the two kinds of evidence, assigns a transparent risk level, and presents the result to an operator through a dashboard.

1.4 Specific Objectives

1. Build a simulated OT environment made of a virtual Modbus TCP PLC, a physical process simulator, sensors and an operator HMI.
2. Normalise network evidence from Zeek, Suricata, Modbus-monitor and operating-system log records, and score it with a network anomaly model.
3. Collect normal telemetry from the simulator and train an Isolation Forest on it as a baseline of normal physical behaviour.
4. Define one normalised event format shared by every component.
5. Implement rule-based correlation of events by asset and time.
6. Implement a documented, transparent risk-scoring method with four risk levels.
7. Provide a web dashboard and a role-based Admin console for monitoring, incident review and audit.
8. Evaluate the physical model using accuracy, precision, recall, F1-score, false-positive rate and detection delay on synthetic scenarios.
9. Keep every component inside a safe, loopback-only, simulation-only boundary.

1.5 Scope and Limitations

In scope. Everything runs on one computer and is bound to 127.0.0.1. The project contains the virtual PLC and process simulator (Part 1), network adapters and a network anomaly model working on replayed fixtures, plus a passive Modbus write monitor attached to the virtual PLC (Part 2), the physical Isolation Forest (Part 3), correlation and risk (Part 4), the OT physical monitor with sensor validation and rules (Part 5), a guarded simulated response (Part 6), a Command Center with a hash-chained audit trail (Part 7), a standalone OT Defense Dashboard (Part 8), an Admin console, an Attack Console for simulated sensor overrides, and launcher scripts.

Out of scope. No real military, industrial, SCADA or PLC equipment is accessed, scanned or modified. No malware or exploit tooling was developed. The abnormal scenarios exist only to test detection.

Limitations.

- The network layer works on replayed, sanitised fixtures. Zeek and Suricata are not run live and no packets are captured. Only the Modbus write monitor observes live requests, and only on the virtual PLC.
- All telemetry is synthetic, so results do not describe real equipment.
- The physical model has a high false-alarm rate and is weak on slow drift (Chapter Four).
- The risk weights and thresholds are experimental design choices and are not safety standards.
- A systematic comparison of network-only, physical-only and fused detection over repeated runs has not been done.
- The system has not been tested against any external or real application.

1.6 Significance of the Project

- **A reproducible laboratory.** A single command starts the services. Students can repeat the experiments without real infrastructure.

- **Evidence, not only alarms.** The project shows how two weak signals can combine into one prioritised incident, and it measures where the individual detectors fail.
- **Transparent decisions.** The risk score is a visible weighted sum. Machine learning supplies one input and never decides alone.
- **Honest reporting.** Results are kept alongside their limits, which makes the work a reliable base for later extension.
- **Safe by design.** Loopback-only binding, an ingest token, CSRF protection, role-based access and an audit trail protect the platform itself.

1.7 Report Organization

Chapter Two reviews OT security, network monitoring, physical anomaly detection, Isolation Forest and event correlation, and states the gap the project addresses. Chapter Three describes the development method, requirements, architecture, workflow, data design and tools. Chapter Four describes the implementation, the testing carried out, the results and the problems met along the way. Chapter Five concludes against the objectives and lists limitations and future work. The appendices contain the repository layout, installation and user guides, configuration details, the test inventory and supporting material.

Chapter Two: Literature Review / Related Work

2.1 Introduction

This chapter reviews the ideas on which CHEM-SHIELD is built: how OT systems are organised and why they are difficult to protect, how network monitoring and physical anomaly detection each work, what is known about the Isolation Forest algorithm, and how evidence from several sources can be correlated and ranked. It ends by stating the gap the project aims to fill.

2.2 Existing Solutions and Concepts

2.2.1 OT Architecture and the Modbus Protocol

A typical OT system has field devices (sensors and actuators), controllers (PLCs), a supervisory layer (HMI or SCADA) and, above them, business networks. The Purdue reference model arranges these into levels, and later work on industrial network segmentation, including the IEC 62443 series, builds on that idea of zones and conduits [2], [4]. Modbus is a simple request–response protocol in which a client reads or writes coils and registers on a server using numbered function codes [3]. Because Modbus carries no authentication, any reachable client that formats a valid request is treated as legitimate. This is the property that makes a protocol-valid but unauthorised write a realistic concern.

2.2.2 OT Security Guidance

NIST SP 800-82 gives guidance for securing OT and explains how its priorities differ from IT: safety and availability of the process rank above confidentiality, and the process itself needs monitoring [1]. IEC 62443 defines security requirements for industrial automation and control systems [2]. Public incident analyses show why this matters. The Stuxnet malware manipulated

industrial control logic while hiding the effect from operators [5], and the 2015 attack on the Ukrainian power grid combined remote access with direct operation of grid equipment [6]. These cases involve physical effects that network logs alone would not have explained. They are cited as motivation only. CHEM-SHIELD does not reproduce them.

2.2.3 Network-Security Monitoring

Zeek produces structured logs of network connections, and Suricata is a signature- and rule-based intrusion detection engine that writes alerts in an EVE JSON format [7], [8]. Both can be deployed passively, so they observe traffic without changing it. For OT they can detect unexpected sources, unusual function codes or bursts of write commands. Their limit, as Chapter One described, is that they judge messages and not the physical outcome of those messages.

2.2.4 Physical-Process Anomaly Detection

Anomaly detection learns what normal data looks like and flags deviations [9]. Surveys of intrusion detection for cyber-physical systems describe approaches based on physical models, statistics and machine learning, and note that attacks on the process are often designed to stay within plausible ranges [10]. Work on stealthy attacks on industrial control systems shows that an attacker who understands a detector can keep the process within the detector’s tolerance for a long time [11]. This is why checking each reading only against a normal range is not enough, and why consistency checks between related variables, for example power against speed, are also used.

2.2.5 Isolation Forest and Feature Extraction

Isolation Forest isolates observations by recursively choosing a random feature and a random split value. Points that are different from the rest need fewer splits to isolate, so they receive shorter average path lengths and higher anomaly scores [12], [13]. The method needs no labelled attacks for training, which suits OT where attack data is scarce, and it is fast enough for a student laboratory. Its limit is the one described above: it scores a point against the learned distribution, so a slow change that stays inside the normal region receives a normal-looking score. In practice the quality of the result also depends on the features. CHEM-SHIELD’s physical model uses the four sensor readings, their per-observation rates of change, and three residuals that compare related variables (Section 4.3.3). scikit-learn provides the implementation used for the physical model [14].

2.2.6 Event Correlation and Risk Assessment

Security information and event management practice groups related events and ranks them so that analysts can look at fewer, better-explained incidents. In the CHEM-SHIELD design, correlation is a deliberately simple rule: events on the same asset whose timestamps are close together belong to one incident. Risk assessment is a weighted combination of the network and physical anomaly scores. A simple weighted sum can be inspected line by line, which is the reason it was preferred to a second learned model. The MITRE ATT&CK for ICS knowledge base provides a shared vocabulary for describing techniques, such as Modify Parameter (T0836) and Manipulation of Control (T0831), and the project uses these as labels for its simulated scenarios [15].

2.3 Limitations of Existing Solutions

1. **Single-source monitoring.** Network tools see messages but not the resulting physical state, and physical monitors see state but not the messages [3], [7].

2. **Point-wise anomaly scores.** Models that score each observation against the normal range are weak on slow or in-range manipulation [10], [11].
3. **Alert volume.** Anomaly detectors flag unusual readings for many harmless reasons, and without ranking the operator carries the burden.
4. **Opaque scoring.** Combining many signals with a learned model makes it difficult to explain why an incident received its priority.
5. **Access to test environments.** Real equipment cannot be used for experiments, and commercial test beds are costly.

2.4 Project Positioning

CHEM-SHIELD does not claim to invent intrusion detection, anomaly detection, SIEM concepts or risk scoring. Its contribution is a small system that joins them in one controlled laboratory and makes the combination measurable.

- **A shared event format.** All components emit events with a timestamp, asset identifier, an event type of exactly `network` or `physical`, a score from 0 to 1, and an evidence object.
- **Asset-and-time correlation.** Events on one asset that are no more than 10 seconds apart are grouped.
- **A visible risk formula.** Network and physical scores are combined with fixed weights and mapped to four levels.
- **Rule-based checks beside machine learning.** Part 5 applies configured limits and physics-consistency rules next to the model score.
- **An operator view with an audit trail.** Incidents are shown in the HMI and in a role-based Admin console, and administrative actions are written to a hash-chained audit log.

Table 2.1 summarises how the project compares with single-source monitoring.

Table 2.1 Comparison of single-source monitoring and CHEM-SHIELD

Capability	Network-only monitoring	Physical-only anomaly detection	CHEM-SHIELD (as implemented)
Sees suspicious communication	Yes	No	Yes (replayed fixtures; live Modbus write monitor on the virtual PLC)
Sees abnormal physical behaviour	No	Yes	Yes
Links the two by asset and time	No	No	Yes (10-second window)
Gives one ranked incident	No	No	Yes (four levels)
Explains the score	Alert text	Anomaly score	Weighted formula and evidence shown
Evaluated on real equipment	Depends on tool	Depends on tool	No (simulation only)

Boundaries of the contribution. The network layer replays fixtures and does not capture live packets. The physical model does not detect slow drift reliably and produces many false alarms on the current data. The risk formula and thresholds are experimental values. The fused comparison over repeated runs is not complete. These points are discussed in Chapters Four and Five.

2.5 Chapter Summary

OT systems rely on protocols with little built-in security, and the two usual monitoring approaches each see only half of an event. Isolation Forest offers a simple, label-free way to model normal physical behaviour but is weak on slow change. A transparent correlation and risk method can combine independent weak evidence into one explained priority. CHEM-SHIELD applies this idea in an isolated laboratory. The next chapter describes how it was designed.

Chapter Three: Methodology and System Design

3.1 Development Methodology and Justification

3.1.1 Approach

The project was built incrementally as numbered parts, each with a clear task and each producing something that could be run and tested before the next part depended on it. Part 1 built the virtual machine, Parts 2 and 3 built the two detection layers, Part 4 joined them, Part 5 added rule checks, Part 6 added the response, Part 7 added the Command Center, and Part 8 added the standalone defense dashboard. The Admin console, Attack Console, shared publisher and launcher scripts were added around these parts.

3.1.2 Justification

1. **The system is a pipeline.** Events flow from detectors to correlation to risk to display. Building the parts in that order let each one be tested against real output from the previous one.
2. **Early working base.** The simulated machine and its telemetry store came first, so the detectors could be trained and tested on data from the same machine.
3. **Safe experimentation.** Each part was kept small enough to be run in isolation, and automated tests were written beside the code.
4. **Honest evaluation.** Training used only normal data, and evaluation used separate scenario data, so results could not be inflated by testing on the training set. Section 4.6 explains how an earlier, too-optimistic figure was replaced.

3.2 Requirements Analysis

3.2.1 Stakeholders and Actors

Table 3.1 Stakeholders and actors

Actor	Role
Operator	Runs the simulated machine in the HMI, sees warnings, may confirm a simulated stop

Analyst	Reviews incidents, acknowledges, escalates, marks false positives, adds notes
Administrator	Manages users, settings and the audit log; has all analyst rights
Tester (Attack Console user)	Sets simulated sensor values to create abnormal conditions for evaluation
Detection components	Network layer, physical ML model, OT monitor, correlation and risk engine

3.2.2 Functional Requirements

Table 3.2 Functional requirements

ID	Requirement	Status
FR1	The system shall simulate a PLC, a motor process and four sensor channels (speed, temperature, pressure, power)	Implemented
FR2	The operator shall be able to start, stop, pause and set a target speed	Implemented
FR3	The system shall normalise network records into a common event format	Implemented on fixtures
FR4	The system shall score network events with an anomaly model	Implemented
FR5	The system shall observe Modbus write requests to the virtual PLC and flag suspicious ones	Implemented (loopback only)
FR6	The system shall train a physical anomaly model on normal telemetry only	Implemented
FR7	The system shall validate sensor data and apply engineering and consistency rules	Implemented
FR8	The system shall correlate events by asset within 10 seconds	Implemented
FR9	The system shall compute a risk score and level for each incident	Implemented
FR10	The system shall store incidents persistently	Implemented (SQLite)
FR11	The system shall show risk and incidents to an operator and to authenticated staff	Implemented
FR12	The system shall offer a simulated stop for a critical condition	Implemented, with a countdown
FR13	The system shall record administrative decisions in an audit log	Implemented, with a hash chain
FR14	The system shall provide guided	Implemented

	scenarios and printable incident reports	
FR15	The system shall ingest live packet captures (Zeek or Suricata running live)	Not implemented
FR16	The system shall compare network-only, physical-only and fused detection across repeated runs	Not done

3.2.3 Non-Functional Requirements

Table 3.3 Non-functional requirements

ID	Requirement	How it is met
NFR1	Safety of scope: nothing may reach real equipment	All services bind to 127.0.0.1; the launcher refuses non-loopback hosts
NFR2	Platform security	Ingest token for event submission, CSRF protection, hashed passwords, rate-limited login, role-based access
NFR3	Transparency	Risk is a visible weighted sum; evidence is stored with every event
NFR4	Reproducibility	Fixed random seeds, saved model, scripted commands
NFR5	Persistence	Telemetry and incidents stored in SQLite and survive restarts
NFR6	Portability	Python project targeting Windows, macOS and Linux
NFR7	Testability	Automated tests for every part

3.2.4 Assumptions and Constraints

- All data is synthetic and the machine is fictional.
- The project had to fit the summer camp timeline, so live packet capture and the fused comparison experiment were not completed.
- The machine is a single asset (`PLC-01`).
- Only open-source tools were used.

3.3 System Architecture

3.3.1 Architectural Style

CHEM-SHIELD is a set of small Python services on one computer. They exchange data through Modbus TCP, HTTP/JSON, a shared SQLite database and JSON files. Detection parts publish scored events to the HMI, which hosts the correlation and risk API. This keeps each part replaceable. For example, the project documentation notes that an OpenPLC runtime could take the place of the project’s own PLC server without changes to the HMI.

3.3.2 High-Level Structure

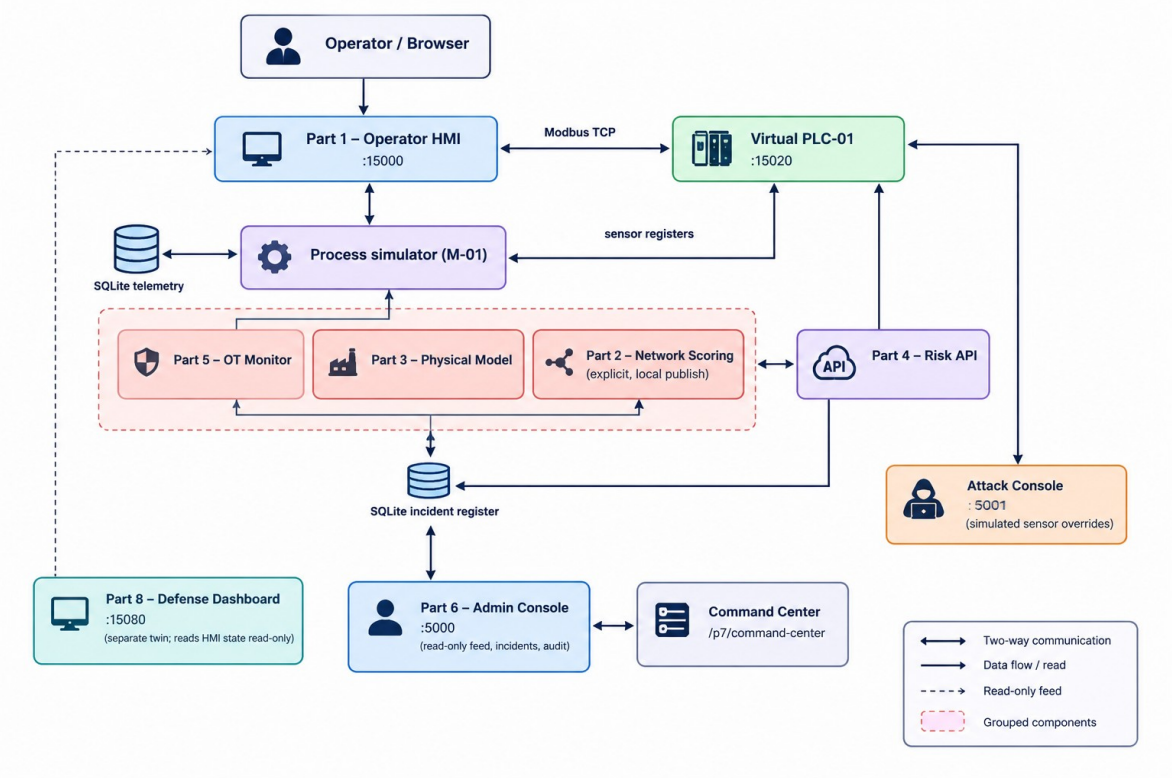


Figure 3.1: High-level architecture of CHEM-SHIELD. [INSERT FIGURE:

3.3.3 Components and Separation of Concerns

Part 1: Virtual OT laboratory. A Modbus TCP server named PLC-01 holds the registers. A separate process simulator reads the commands, advances a simple process model, writes sensor values back to the PLC and stores telemetry in SQLite. A Flask HMI lets the operator control the motor.

Part 2: Network evidence. Adapters convert Zeek connection logs, Suricata EVE records, passive Modbus-monitor records and exported operating-system security logs into one schema. A network anomaly model trained on a normal baseline gives each record a bounded score. In addition, a passive write monitor inspects decoded Modbus write requests arriving at the virtual PLC.

Part 3: Physical machine learning. An Isolation Forest is trained on normal telemetry only and outputs a binary decision plus a separate score between 0 and 1.

Part 4: Correlation and risk. Events are validated, de-duplicated, grouped by asset and time, scored with the risk formula, and stored as incidents.

Part 5: OT monitor. It validates sensor data, applies configured limit and consistency rules, and can score readings with the saved Part 3 model.

Part 6: Response. It provides the warning pop-up and the simulated stop, which requires operator confirmation or follows a countdown.

Part 7: Command Center. It provides guided simulation scenarios, a timeline, an audit-integrity check and printable reports.

Part 8: OT Defense Dashboard. A separate application with its own reactor model, protocol scenarios, rules, Isolation Forest, Purdue-zone policy and report export.

Admin console and Attack Console. The Admin console is the supervisory and navigation hub. The Attack Console sets simulated sensor values for testing.

Shared publisher. A loopback-only component that posts scored events to the HMI with an ingest token.

Table 3.7 Services and ports

Service	Address	Role
Virtual PLC	127.0.0.1:15020	Modbus TCP register server
Process simulator	(client of PLC)	Process model, telemetry writer
Machine HMI	127.0.0.1:15000	Operator screen, risk API
Admin console	127.0.0.1:5000	Incidents, audit, users, Command Center
Attack Console	127.0.0.1:5001	Simulated sensor overrides
OT Defense Dashboard (optional)	127.0.0.1:15080	Standalone Part 8 dashboard

3.3. Integration View

Detectors do not call each other. P2 and P5 publish scored events to the HMI's `POST /api/risk/events` endpoint, and only when started with the explicit `--publish` option. No background capture is active by default. Part 8 reads the HMI's `/api/state` and `/api/risk` endpoints with GET requests only and never sends commands.

3.4 System Design and Workflow

3.4.1 Data Flow

The offline chain runs one stage after the other and passes one JSON file between stages.

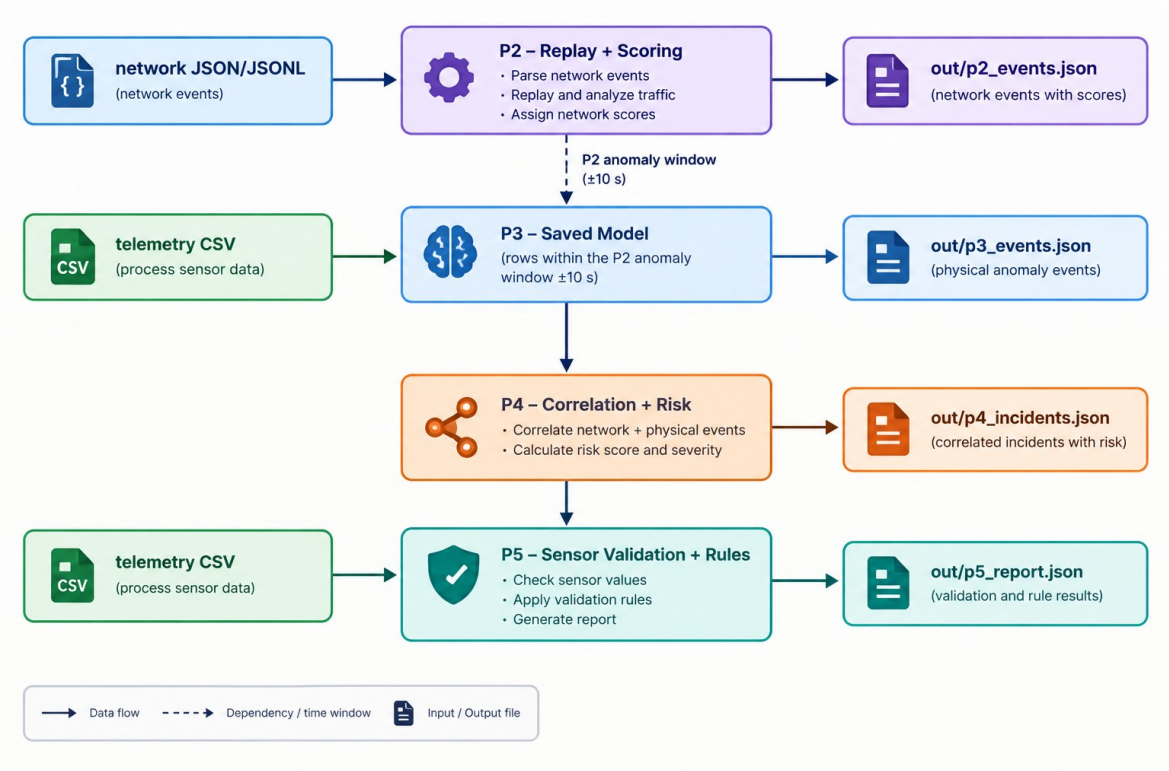


Figure 3.2: Data flow of the staged P2 to P5 chain.

For the live machine, P5 can follow newly appended telemetry in the SQLite database and publish debounced events to the HMI, where Part 4 correlates them with network events.

3.4.2 Correlation and Risk Decision Flow

1. A scored event is submitted. The validator checks that the type is `network` or `physical` and that the score is finite and between 0 and 1.
2. Duplicate event identifiers are dropped.
3. If an active incident exists for the same asset and the event is within 10 seconds of its latest event, the event joins that incident. Otherwise a new incident starts.
4. For each type, the highest available score in the incident is used.
5. The risk engine computes the score and level.
6. The incident is saved and shown. Acknowledging an incident is recorded separately and never lowers its risk.

An incident whose events carry no score is marked UNASSESSED. The pipeline does not invent a zero.

Table 3.6 Risk levels and thresholds

Final risk	Level
below 0.30	LOW
0.30 to below 0.60	MEDIUM
0.60 to below 0.80	HIGH
0.80 or above	CRITICAL

The weights must add up to 1.0 and the thresholds must increase, otherwise the engine rejects the configuration. The scores are anomaly indicators and are not probabilities. The weights and thresholds are experimental values chosen for the project and are not industrial or safety standards.

3.4.3 Machine Condition Band

Separately from the incident score, the Admin dashboard shows a live machine band based on the highest ratio of a reading to its configured limit. Below 75% of a limit the band is NORMAL, from 75% up to the limit it is MEDIUM, and above any limit it is CRITICAL. The limits are 70 °C, 4 bar, 12 kW and 1500 RPM. This indicator is not a probability and is independent of the detector score.

3.4.4 Worked Example

An operator or tester forces the temperature above its limit while a suspicious write arrives at the PLC. The write monitor emits a network event, and the OT monitor or physical model emits a physical event on the same asset a few seconds later. Both fall in the 10-second window, so one incident is created. Using the project's default weights, a network score of 0.9 alone gives 0.36 (MEDIUM), a physical score of 0.9 alone gives 0.54 (MEDIUM), and both together give 0.90 (CRITICAL). These figures were calculated with the project's risk engine from illustrative inputs. They are not measured detector scores. Section 4.6 shows a calculation with scores produced by the actual fixtures.

3.5 Data and Database Design

3.5.1 Storage Strategy

The project uses SQLite files and JSON files. No separate database server is required.

3.5.2 Data Stores

Machine telemetry database (`chem_shield_p1/data/telemetry.db`):

Table	Main content
telemetry	id, timestamp, machine_id, plc_id, motor_status, target_rpm, rpm, temperature, pressure, power
events	id, timestamp, source, machine_id, event_type, parameter, old_value, new_value, severity
risk_incidents	incident_id, asset_id, start_time, end_time, status, risk_level, risk_score, risk_assessment_json, events_json, acknowledged_at, acknowledged_by, created_at, updated_at
risk_events	event_id, incident_id, timestamp, source, event_json
risk_ingest_state	last received and accepted times, accepted and rejected counts

Admin database: `incident_status` (incident_id, status, updated_by, updated_at, note) and `audit_log` (id, timestamp, username, role, source_ip, action, incident_id, detail). Part 7 adds `p7_audit_chain` (chain_order, audit_id, prev_hash, hash, record_json).

Other local files: `admin_part/data/users.json` (usernames, roles, password hashes; excluded from the repository), `chem_shield_p3/models/isolation_forest.joblib`, versioned CSV data and result files under `chem_shield_p3/`, and Part 8 reports and logs under `chem_shield_p8/data/`.

3.5.3 Virtual PLC Register Map

Table 3.4 Virtual PLC holding-register map (slave ID 1, zero-based, 13 registers)

Address	Name	Meaning
0	Motor command	0 STOP, 1 RUN (HMI to PLC)
1	Target RPM	Requested speed
2	Actual RPM	Written by the process simulator
3	Temperature	°C, stored as integer hundredths
4	Pressure	bar, integer hundredths
5	Power	kW, integer hundredths
6	Event code	0 none, 1 rpm deviation, 2 temperature increase, 3 pressure spike, 4 sensor inconsistency
7	Event value	Value associated with the event
8	Override mask	Bits for temperature (1), pressure (2), power (4), speed (8)
9 to 12	Override values	Simulated temperature, pressure, power, speed set by the Attack Console

3.5.4 Normalised Event Format

Table 3.5 Normalised event format

Field	Description
event_id	Unique identifier; duplicates are dropped
timestamp	ISO-8601, normalised to UTC
source	Producer name, for example `zeek`, `suricata`, `ot_monitor`, `physical_ml`
asset_id	Asset the event concerns, for example `PLC-01`
event_type	Exactly `network` or `physical`
anomaly_score	Optional number from 0 to 1 (not a probability)
severity	Optional label
evidence	Source-specific details kept with the event

3.6 Tools, Technologies, and Development Environment

3.6.1 Technology Stack

Table 3.8 Technology stack

Category	Technologies and role
Language	Python (3.10 or newer recommended; project documentation targets 3.9 to 3.13)
Web	Flask 3 (>=3.0, <4) for HMI, Admin console, Attack Console and Part 8; Flask-WTF for CSRF protection; Werkzeug 3 for password hashing
Industrial protocol	pymodbus (>=3.6, <3.8) for Modbus TCP server and client
Machine learning	scikit-learn (>=1.3, <1.7), NumPy (>=1.24), pandas (>=2.0), joblib (>=1.3)
Network model	Isolation Forest implemented in pure Python in `chem_shield_p2/network_isolation_forest.py`
Configuration	PyYAML (>=6.0, <7) for Part 8
Reports	reportlab (>=4.0, <5) for PDF output from Part 8; Matplotlib (>=3.7) for ML graphs
Storage	SQLite (Python standard library) and JSON/CSV files
Testing	pytest (>=8.0)
Front end	HTML, CSS and JavaScript served by Flask; Server-Sent Events in Part 8

The physical model file records that it was trained with scikit-learn 1.6.1. In our verification run the dependencies resolved to Flask 3.1.3, Flask-WTF 1.3.0, pymodbus 3.7.4, scikit-learn 1.6.1, NumPy 2.5.4, pandas 3.1.0, reportlab 4.5.1 and pytest 9.2.0 on Python 3.12.3. [INFORMATION REQUIRED: add the Python version and operating system used on your own machines.]

3.6.2 Justification of Key Choices

- **Python throughout.** One language covers simulation, machine learning, web interfaces and tests, which suited the team and the timeline.
- **Isolation Forest.** It needs no labelled attacks, is light enough for a laptop and is well documented [12], [14].
- **Rule-based correlation and a weighted score.** They are simple to explain, and every number in an incident can be traced.
- **SQLite.** It needs no server, which keeps the laboratory easy to start and to reproduce.
- **Loopback-only design.** It makes sure that nothing in the project can reach a real network by accident.

3.6.3 Development Environment

The team developed on laptops with a project virtual environment. The repository includes a launcher (`scripts/chemshield.py`) with `setup`, `test`, `run`, `stop` and `status`

commands, a one-command demo launcher (`scripts/run_all.py`), and a stop script. Source code was kept in a Git repository. [INFORMATION REQUIRED: hosting service and team workflow.]

3.7 Chapter Summary

This chapter described the incremental, pipeline-ordered development approach, the requirements and the status of each, the architecture, the event, risk and data designs, and the tools. The next chapter describes what was implemented and how it was tested.

Chapter Four: Implementation and Results

4.1 Introduction

This chapter explains how the design was implemented, states what is complete and what is not, and reports the test and evaluation results. Everything reported here comes from the source code and result files in the project, and from a verification run described in Section 4.5. Where a result must come from your own live run or from the demonstration, it is marked as pending.

4.2 Implementation Status Summary

Table 4.1 Implementation status of each component

Component	Status
Virtual PLC, process simulator, HMI (P1)	Implemented
Attack Console	Implemented
Network adapters (Zeek, Suricata, Modbus monitor, OS log)	Implemented on fixtures
Network anomaly model	Implemented
Live Modbus write monitor	Implemented
Physical Isolation Forest (P3)	Implemented
Correlation and risk engine (P4)	Implemented
OT monitor rules and validation (P5)	Implemented
Warning pop-up and operator-confirmed stop (P6)	Implemented
Auto-response module with recovery and auto-resume (`auto_response.py`)	Partially implemented
Admin console with roles and audit	Implemented
Command Center, hash-chained audit, reports (P7)	Implemented
OT Defense Dashboard (P8)	Implemented

4.3 Implementation Details

4.3.1 Part 1: Virtual PLC, Process Simulator and HMI

The PLC server (`chem_shield_p1/plc/modbus_plc.py`) exposes the register map in Table 3.4. The process simulator (`process/machine.py`) models a fictional motor as a set of first-order responses. At each step of 0.5 seconds the speed moves toward the target, and temperature, pressure and power move toward values that depend on the load, which is speed divided by 1500. The target values are `20 + 47 × load` for temperature, `1 + 2.8 × load` for pressure and `0.2 + 11.6 × load` for power. Random noise is added, and the normal model is capped below the limits (for example 68.9 °C, 3.95 bar, 11.9 kW and 1490 RPM), so normal operation fluctuates below the limits and does not rest on them. The random generator is seeded (42) for repeatability.

The HMI (`hmi/app.py`) provides START, STOP, PAUSE and APPLY TARGET controls, the live machine view, command history, and the API used by the other parts. The process-condition badge reads CRITICAL when temperature exceeds 70 °C, pressure exceeds 4 bar, power exceeds 12 kW or speed exceeds 1500 RPM.

4.3.2 Part 2: Network Evidence

The adapters (`chem_shield_p2/adapters.py`) convert fixture records to the normalised format, using a hash of the source, time and evidence as a stable event identifier. The network model (`network_isolation_forest.py`) is an Isolation Forest written for the project, with 64 trees and a depth limit of 8. It is trained on the normal rows of `fixtures/network_baseline.json`, using features taken from the evidence such as duration, packet and byte counts, ports, connection state and Modbus fields, and its scores are calibrated to the range 0 to 1. A configuration file maps fixture endpoint identities (for example `10.10.10.5`) to the asset `PLC-01`.

The passive Modbus write monitor (`modbus_write_monitor.py`) inspects decoded write requests on the virtual PLC. It never blocks or changes a request. It raises events for the cases in the table below, and repeats of the same rule from the same source are suppressed for 10 seconds.

Rule	Condition	Severity	Score
unknown_modbus_source	Write from an address not on the allow-list (default 127.0.0.1)	high	0.90
unexpected_register_write	Write outside the expected register range	high	0.85
unusual_write_function_code	Write function code other than 6 or 16	high	0.80
simulation_override_write	Write to the override registers by the Attack Console	medium	0.70
modbus_write_burst	5 or more writes from one source within 1 second	medium	0.75

4.3.3 Part 3: Physical Isolation Forest

The detector (`chem_shield_p3/src/detector.py`) wraps scikit-learn's `IsolationForest` with a fixed random state of 42 and a contamination setting of 0.05. It is trained only on normal readings generated by the Part 1 process simulator: 6,000 rows covering start-up, stopping and several target speeds from 0 to 1450 RPM. Training is refused if the data contains a label other than normal.

Eleven features are used: the four readings (speed, temperature, pressure, power), their four per-observation rates of change, and three residuals that compare related quantities (power against speed, temperature against power, and pressure against speed). The residuals help a value that is inside its normal range but wrong for the current operating state to stand out. The first reading of an independent stream receives zero rates. Short gaps in the data are filled by interpolation.

The binary decision comes directly from the Isolation Forest. The separate `anomaly_score` is a calibrated value from 0 to 1 and is not used to make the decision and is not a probability. The model is saved as `models/isolation_forest.joblib`, together with its feature list and settings, and loaded without retraining.

4.3.4 Part 4: Correlation and Risk

The correlation engine and incident store (`chem_shield_p4/correlation/`) implement the flow in Section 3.4.2, and the risk engine (`chem_shield_p4/risk_engine/`) implements the formula in Section 3.4.3 with a validated configuration. The HMI exposes these functions through the following routes:

- `POST /api/risk/events` to submit events
- `GET /api/risk` for the highest active risk
- `GET /api/incidents` and `GET /api/incidents/<id>` for history and detail
- `POST /api/incidents/<id>/acknowledge` to record an acknowledgement

Event submission requires the `X-Ingest-Token` header. If no token is set, the HMI generates a random one for that run.

4.3.5 Part 5: OT Monitor

The monitor (`chem_shield_p5/ot_monitor/`) validates each reading (range and staleness checks) and then applies configured rules. A rule is active only where a threshold is set in the configuration. The example configuration sets upper limits of 70 °C, 4 bar, 12 kW and 1500 RPM, each with CRITICAL severity, and defines these consistency checks:

- speed rising by 150 RPM or more while power changes by 0.25 kW or less
- temperature falling by at least 1 °C while power is at least 4 kW
- pressure changing by less than 0.1 bar while speed changes by 200 RPM or more
- a sensor frozen at one value for 30 seconds while running

If the model score reaches 0.5, a physical anomaly event is also created. If a reading cannot be scored, the monitor emits a medium-severity warning event instead. The monitor can read a CSV file, read the telemetry database, or follow new rows continuously, and it publishes only when `--publish` is given.

4.3.6 Part 6: Simulated Response

When a fresh CRITICAL incident or a live limit violation appears, the HMI shows a warning. The operator can acknowledge it and, for a critical incident, confirm an immediate simulated stop. The request carries a page-generated confirmation token and is accepted only when the caller and the configured PLC address are loopback addresses. An automatic simulated stop timer is also armed, with a default of 900 seconds, which can be changed with the environment variable `CHEM_SHIELD_AUTO_STOP_SECONDS`. If readings return to normal, the timer is cancelled, and readings that stay inside the limits for 30 seconds resolve the related incidents. Historical or replayed events cannot raise warnings or arm the timer. The machine does not restart automatically.

The repository also contains `chem_shield_p6/auto_response.py`, a state machine with a 15-second countdown, a recovery check and an optional automatic restart. It is covered by 14 unit tests but is not connected to the HMI, and a test confirms that the HMI does not expose an automatic-response endpoint. It is therefore reported as partially implemented.

4.3.7 Admin Console

The Admin console (`admin_part/app.py`) requires login. Accounts are created from the command line, and only password hashes are stored. Three roles exist: administrator (incidents, decisions, users, settings, audit log), analyst (view, acknowledge, escalate, mark false positive, add notes, record a simulated response) and operator (read-only view of incidents affecting the process). The login is rate-limited, forms use CSRF tokens, and a failed-login detector raises a HIGH alert for five failed attempts in five minutes. The dashboard polls the HMI every three seconds and shows OFFLINE or STALE when data is missing or older than ten seconds. An acknowledgement is the only write to the HMI. All other decisions are local audit records and are never executed against the machine.

4.3.8 Attack Console

The Attack Console (`the_attack/app.py`) accepts a numeric value for temperature (0 to 120 °C), power (0 to 20 kW), pressure (0 to 10 bar) or speed (0 to 3000 RPM) and writes it to the override registers. The value stays active until changed or cleared. Requests are audited, and the history stays pending until later telemetry confirms the value. When a measured value exceeds a configured limit, the console publishes a risk event to the HMI. It does not control the motor.

4.3.9 Part 7: Command Center

The Command Center (`/p7/command-center`, available after Admin login) offers five guided scenarios that use only the Attack Console API:

Scenario	Simulated steps
Overheat	Temperature set to 92 °C
Pressure spike	Pressure set to 5.2 bar
Overspeed	Speed set to 1800 RPM
Stealthy in-range drift	Temperature set to 48, then 57 at 8 s and 66 °C at 16 s, all below the limit
Cascade	Temperature 82 °C, pressure 4.8 bar at 25 s, power 14 kW at 55 s

The runner clears all overrides when a scenario ends or is aborted. The scenarios carry labels from MITRE ATT&CK for ICS (Manipulation of Control T0831, Modify Parameter T0836, tactic Impair Process Control TA0106) as illustrations of simulated parameter changes [15].

An additional table, `p7_audit_chain`, stores a SHA-256 hash of each audit record chained to the previous one, so that any later change to an audit row can be detected on the integrity page. A helper script modifies only a copy of the database to demonstrate this. A 0 to 100 “security posture” demo score is the sum of four equal parts of 25 points (detection speed, operator response, automatic stop, audit integrity). The project documentation describes it as a simple demo metric and not a standard. Critical events are written to an on-screen outbox and a local log. Telegram delivery is optional and needs the user’s own credentials. Printable HTML incident reports can be saved as PDF from the browser.

4.3.10 Part 8: OT Defense Dashboard

Part 8 (`chem_shield_p8/`) is a separate application on port 15080. It contains a reactor model with an in-memory PLC map, a physics-based guard (rate, energy, mass and cross-sensor rules) with its own Isolation Forest trained on locally generated reactor data (11 features, contamination 0.03, 8 simulated runs of 180 steps), a signed-baseline and hash-chain watch over safety-system settings, a Purdue-zone policy check using documentation-range IP addresses, and six replay scenarios: setpoint manipulation, sensor replay, rogue engineering workstation, loss of view, safety bypass attempt and IT-to-OT lateral movement. Response playbooks only take effect after explicit operator approval and change only the dashboard’s own state. The dashboard can export HTML and PDF reports. It reads the HMI through GET requests and sends nothing to the machine. Its reactor twin is a different data source from the Part 1 machine.

4.3.11 Launchers and Scripts

`scripts/chemshield.py` provides `setup`, `test`, `run`, `stop` and `status`. `run` starts the PLC, process simulator, HMI, Admin console and Attack Console in dependency order and checks that their ports are free. `scripts/run_chain.py` runs the staged P2 to P5 chain, and `scripts/run_pipeline_demo.py` publishes fixture events to the HMI.

4.4 Major Features and Interfaces

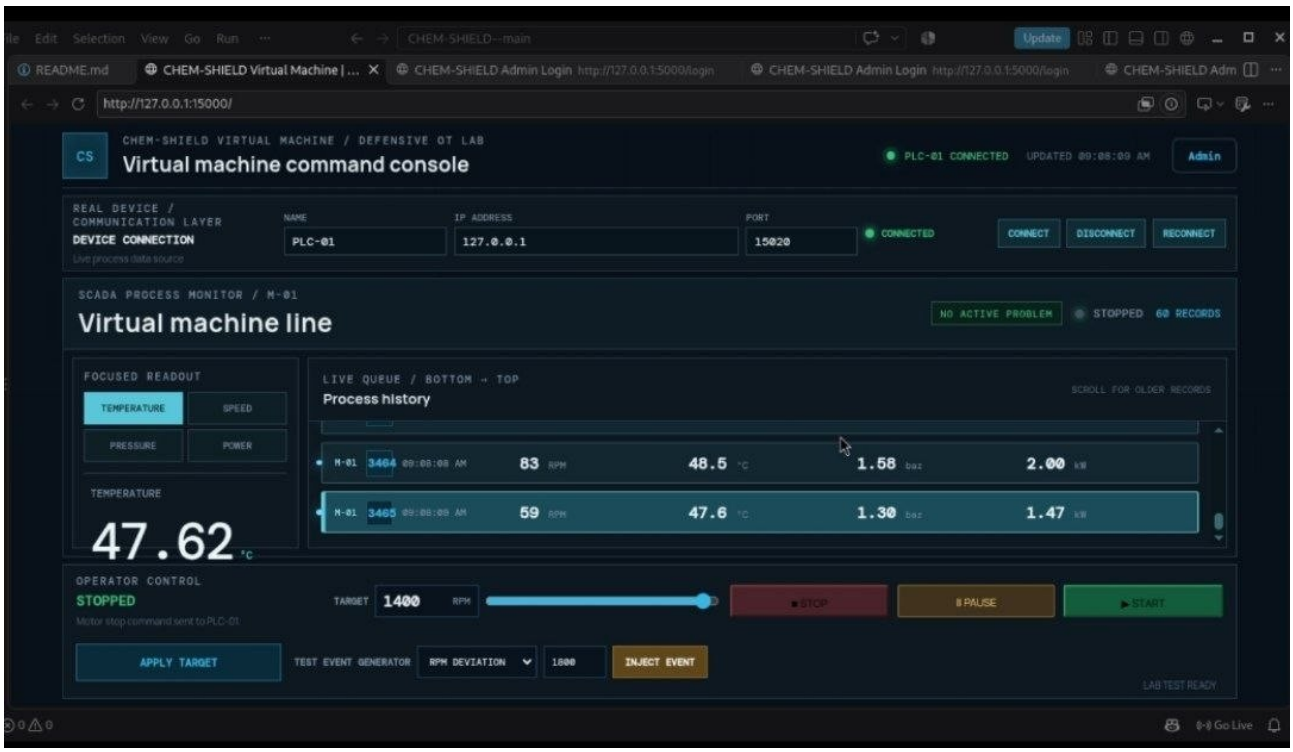


Figure 4.1: Operator machine HMI. Shows the PLC connection, speed, temperature, pressure and power, the START/STOP/PAUSE/APPLY TARGET controls, history and the machine-condition badge.

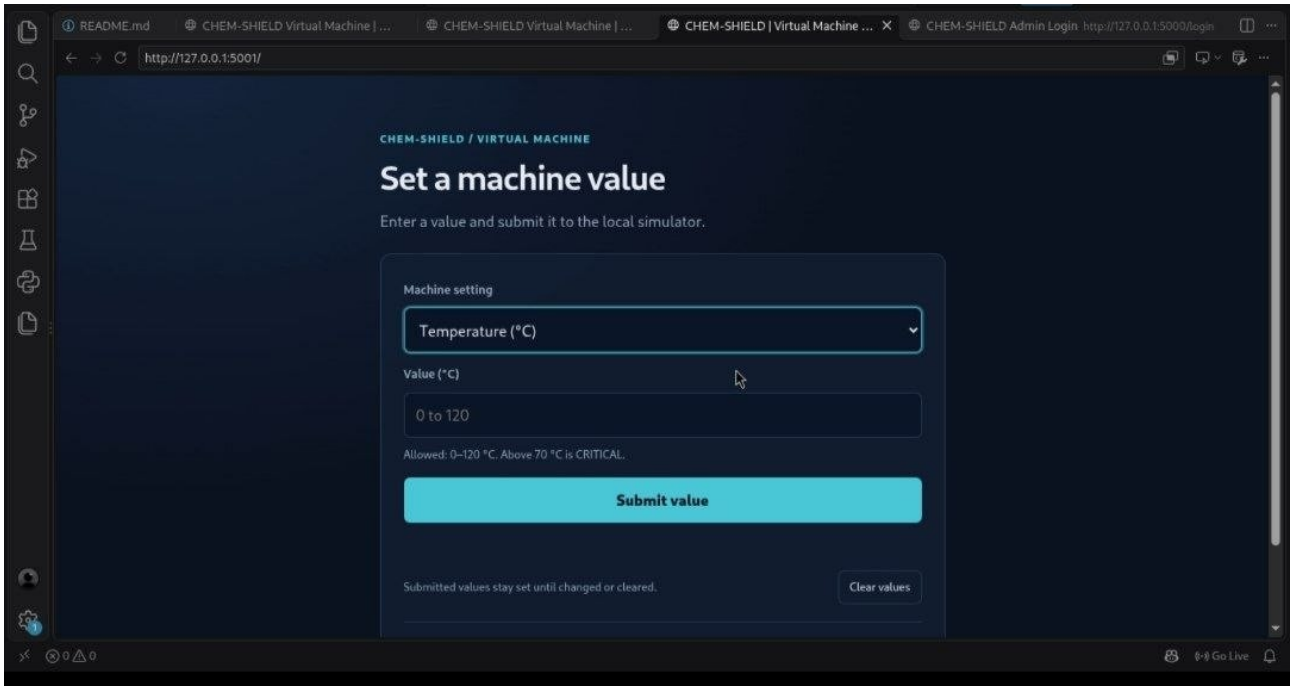


Figure 4.2: Attack Console.

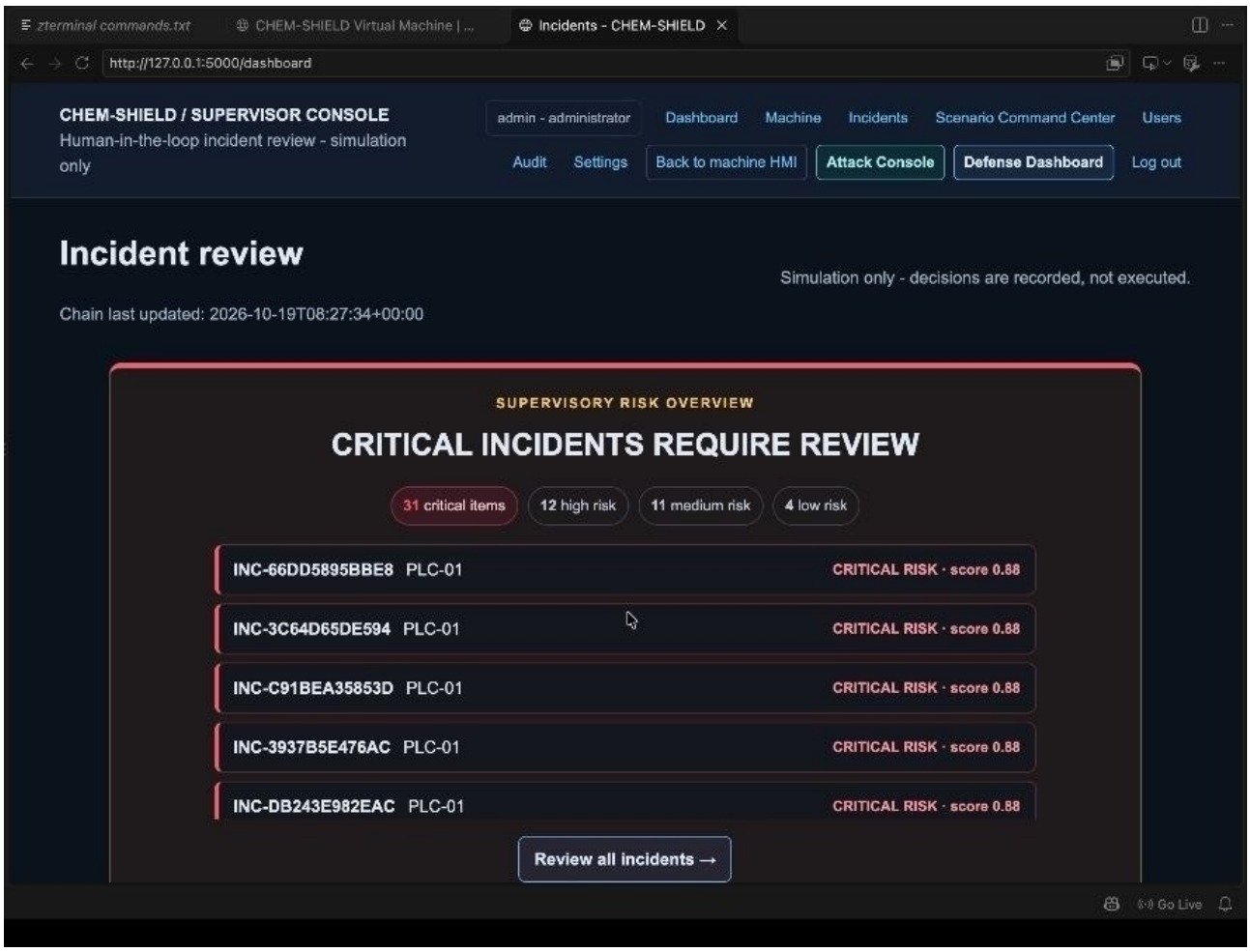


Figure 4.3: Admin console dashboard. Shows the live machine band, incident counts and detector risk.

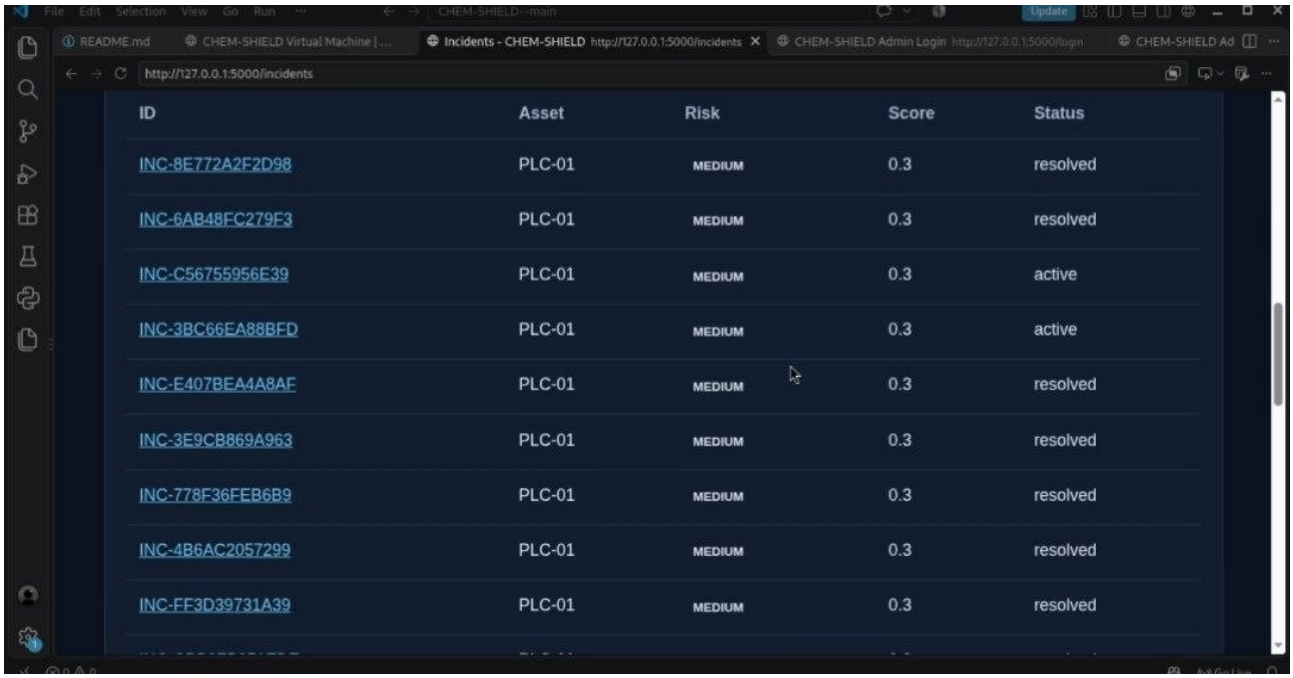


Figure 4.4: Incident detail page. Shows the correlated events, their scores, the risk calculation and the analyst actions.

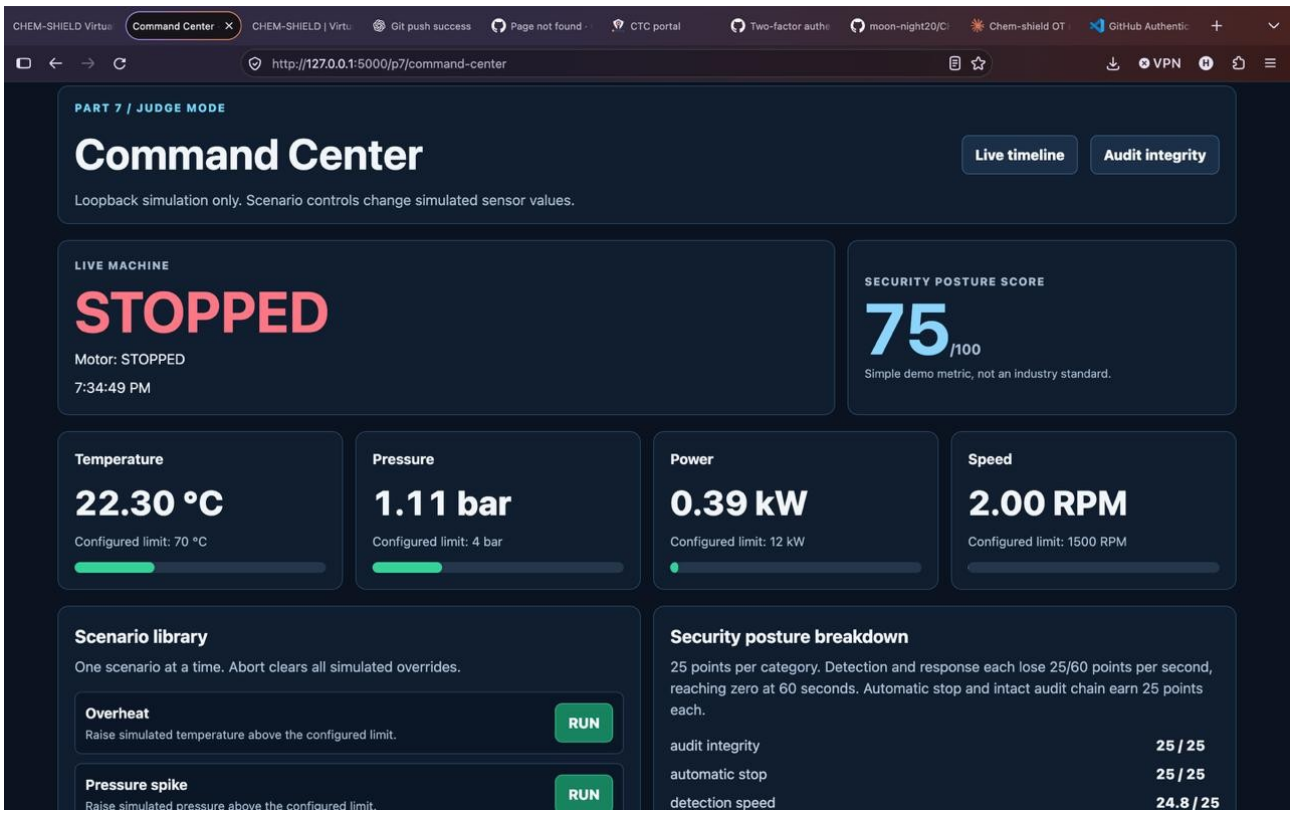
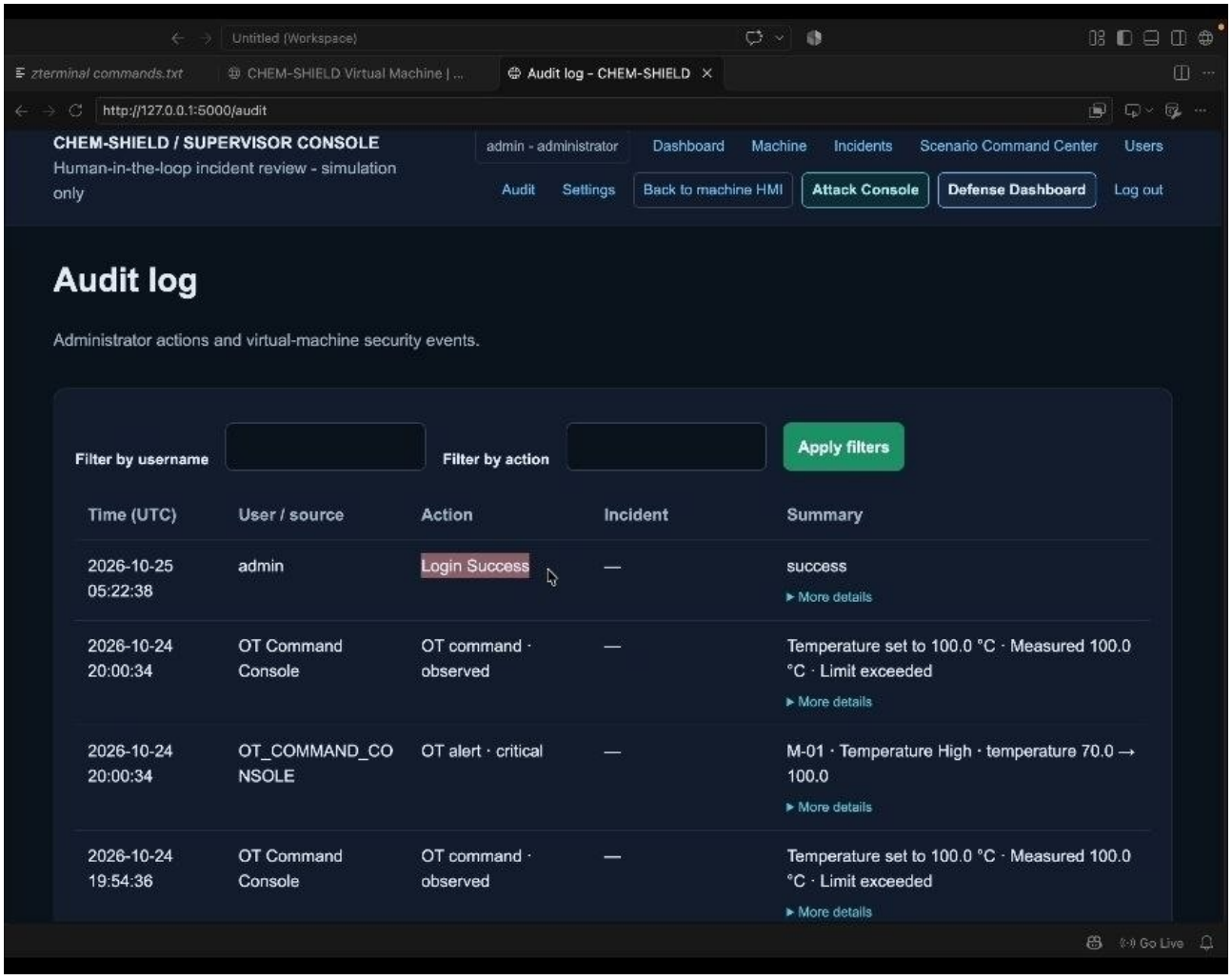


Figure 4.5: Command Center. Shows the scenario buttons, countdown, timeline and posture score.



- **Figure 4.6: Audit integrity page.** Shows the verification result of the hash chain.

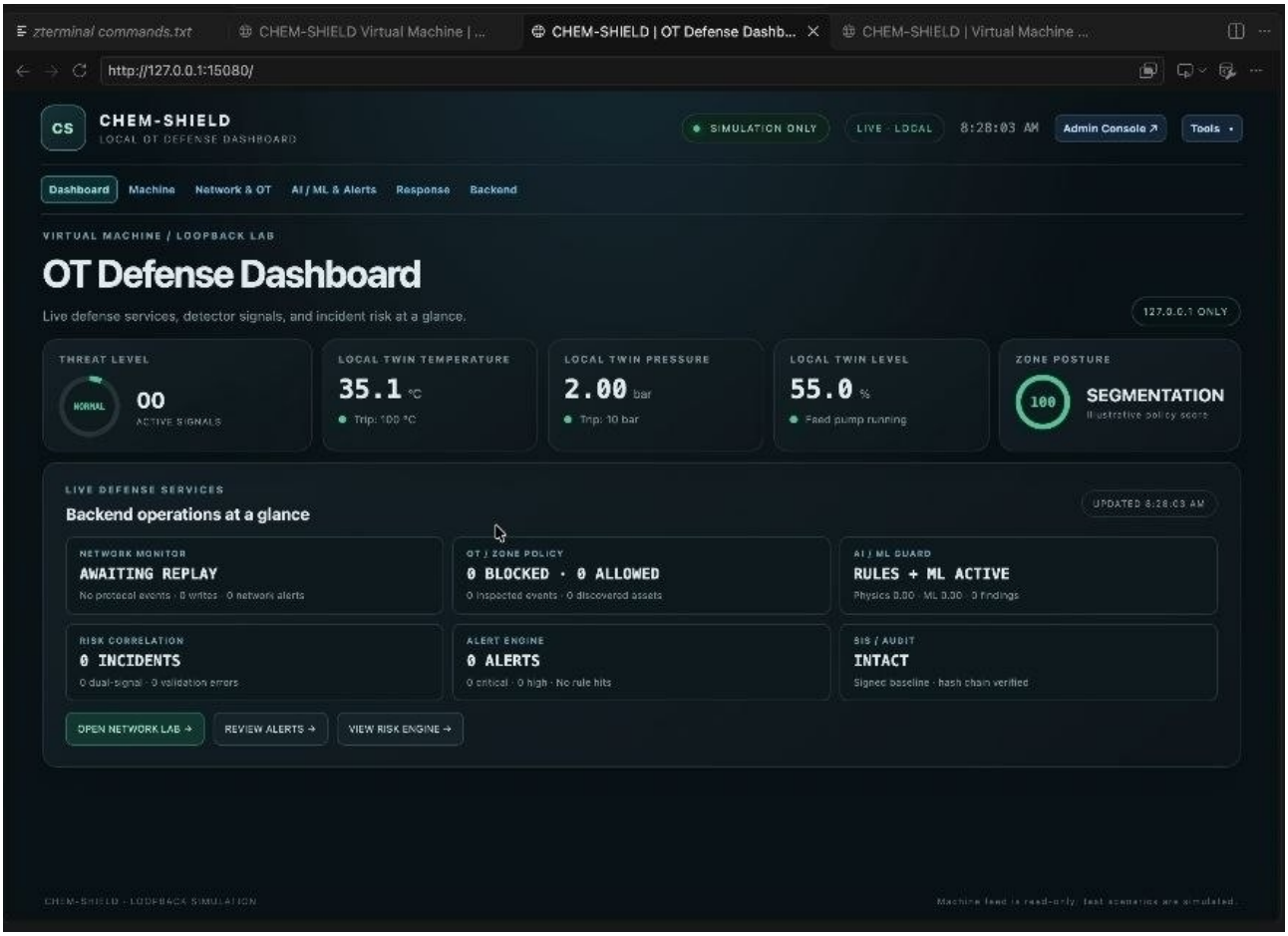


Figure 4.7: OT Defense Dashboard. Shows the data-path panel, reactor twin, scenario playback and alerts.

4.5 Testing Methods and Results

4.5.1 Testing Strategy

Table 4.2 Testing strategy and targets

Level	Tool	Target
Unit	pytest	Risk formula, correlation, validators, adapters, network model, feature engineering, rules, response state machine, hash chain
Integration	pytest with the Flask test client	HMI routes, ingest token, risk API, Admin console access and CSRF, Attack Console, launchers
Staged chain	`scripts.run_chain`	Offline P2 to P3 to P4 to P5 hand-offs
ML evaluation	`chem_shield_p3.scripts.evaluate_heldout`, `evaluate_model`	Detection on synthetic scenarios

Scenario	Command Center and Attack Console	Live guided scenarios (pending, see Table 4.6)

4.5.2 Automated Test Results

The full test suite was run from the repository root with `python -m pytest`. The run was done by us on a Linux container with Python 3.12.3 while preparing this report. The result was **351 passed, 17 subtests passed** in 113.87 seconds, with no failures. Table 4.3 shows how the tests are distributed.

Table 4.3 Automated test results by area

Area	Tests	Result
Admin console	49	Passed
Shared publisher	4	Passed
Part 1 (HMI, PLC, process)	35	Passed
Part 2 (network)	14	Passed
Part 3 (physical ML)	7	Passed
Part 4 (correlation and risk)	25	Passed
Part 5 (OT monitor)	22	Passed
Part 6 (response)	19	Passed
Part 7 (Command Center)	18	Passed
Part 8 (Defense Dashboard)	112	Passed
Scripts and chain	22	Passed
Attack Console	24	Passed
Total	**351**	**All passed**

The tests cover, among other things, event validation, duplicate suppression, the 10-second grouping, the risk formula and its configuration checks, rejection of events without the ingest token, loopback-only binding, Admin role and CSRF controls, the Modbus write monitor, audit hash-chain verification, and the Part 8 modules and end-to-end runs of each synthetic scenario. Appendix E lists the test files.

4.5.3 Physical Model Evaluation Method

The evaluation scripts (`evaluate_heldout.py`, `evaluate_model.py`) train a fresh model on the normal P1-simulated dataset only and score two kinds of data: a separate one-hour normal run of 7,200 readings at 0.5 seconds each, and generated attack scenarios. The scenarios stealthy_in_range and slow_gradual_drift never appear in training. We re-ran both scripts in a scratch copy of the repository and obtained exactly the figures stored in `chem_shield_p3/results/`.

4.6 Evaluation, Results, and Discussion

4.6.1 Physical Model Results

Over all 7,420 synthetic observations, the model gave an accuracy of 0.937, a precision of 0.215, a recall of 0.423, an F1-score of 0.285 and a false-positive rate of 0.047 (TP 93, TN 6,861, FP 339, FN 127). Detection delay could be measured for 5 of the 6 attack scenarios. The mean delay was 8.0 samples, the median was 1 sample and the maximum was 34 samples. One sample is 0.5 seconds.

Table 4.5 Physical ML detection results by scenario

Scenario	Anomalous samples	Detected	Detection rate	First detection
Normal (7,200 readings)	0	339 false alarms	FPR 4.71% (339 per hour)	n/a
sudden_change	30	0	0.0%	not detected
unstable_behavior	30	19	63.3%	2.0 s
gradual_deviation	40	4	10.0%	17.0 s
heavy_sensor_noise	40	32	80.0%	0.5 s
slow_gradual_drift	40	3	7.5%	0.5 s
stealthy_in_range	40	35	87.5%	0.0 s

Source: `chem_shield_p3/results/scenario_evaluation.csv` *and* `heldout_scenario_evaluation.csv`.

Discussion. The model detected stealthy in-range samples well (87.5%). This is partly because the consistency residuals make a value that does not fit the current operating state stand out. It detected slow drift and gradual deviation poorly. The result for `sudden_change` needs a comment. None of the 30 samples was flagged, and the data file shows that these samples stay within the range covered by normal operation (speed about 877 to 1,101 RPM, temperature about 47.6 to 54.9 °C). We observed this in the data, but we did not investigate the cause further. It suggests that the scenario generator and the simulator's normal envelope are not well matched. [INFORMATION REQUIRED: confirm with the team whether `sudden_change` was meant to leave the normal envelope.] The false-alarm rate is high for a real alert stream, because 339 alarms in one hour of normal operation would overwhelm an operator. This result is the main reason the model is treated as one piece of evidence inside the correlation method and does not decide alone.

All of these figures come from synthetic data generated by the project. They show how the pipeline behaves and are not estimates of performance on real equipment. An earlier figure of 97% accuracy and a 4% false-positive rate came from 200 samples generated by the same generator for training and testing. It is kept in the repository as a historical record only and is not used here.

4.6.2 Scenario Results

Table 4.4 Scenario results: normal, network-only, physical-only, correlated

Scenario	Input	Expected result	Actual observed result	Status
Normal operation (network)	Two benign fixture records (Modbus-monitor and OS-log)	Low score, no incident	Both scored 0.003, below the 0.5 threshold	Passed
Normal operation (physical)	7,200 normal readings, one simulated hour	Few alerts	339 flagged (4.71%)	Observed, high false-alarm rate
Network-only	Zeek record (score 0.996) and Suricata alert (score 0.999) on PLC-01, no physical event	MEDIUM	Network contribution 0.400, final risk 0.400, MEDIUM	Passed
Physical-only	One physical model event, score 0.725, no network event	MEDIUM	Physical contribution 0.435, final risk 0.435, MEDIUM	Passed
Correlated	The network events above plus the physical event 5 to 6 seconds later	Higher than either alone, CRITICAL	One incident INC-0001, risk 0.835, CRITICAL	Passed
Rule confirmation	P5 applied to the incident window	Rule agrees with the model	`temperature_high` at 99.0 °C against a 70 °C limit, severity critical; rule-confirmed	Passed

The scores in this table come from the project's own fixtures and saved models, run through the staged chain (`scripts.run_chain --step-by-step`) in a scratch copy, and from the risk engine. The final risk values were computed with the project's `RiskEngine` using those scores.

The chain output reported: P2 four events, two above the threshold of 0.5; P3 two telemetry rows scored with one physical anomaly; P4 one incident, highest risk CRITICAL (0.835); P5 one rule-confirmed incident. These scores come from fixture data with fixed dates in October 2026 and are demonstration values, not live detections.

The result supports the central observation of the project: each source alone stays at MEDIUM, and only the correlated pair reaches CRITICAL. The comparison is based on one fixture scenario. It does not show how often this holds in general.

4.6.3 Pending Evidence

Table 4.6 Test cases that remain pending

Test case	What is needed	Status
Live normal operation on the running machine	Start all services, run at a normal target, record that no incident appears	Pending
Live network-only scenario	Trigger a Modbus write monitor event (for example an Attack Console override write) and record the HMI and Admin result	Pending
Live physical-only scenario	Set a sensor override above its limit and record the limit alert, incident and countdown	Pending
Live correlated scenario	Combine a monitor event and a physical event within 10 s on the live HMI	Pending
Guided scenarios (Overheat, Cascade and so on) with screenshots	Run from the Command Center	Pending
Automatic simulated stop with a short countdown	Set `CHEM_SHIELD_AUTO_STOP_SECONDS=20` and record the result	Pending
Audit tamper demonstration	Run `scripts/p7_tamper_demo.py` and verify the integrity page	Pending
Latency or load measurement	Not measured; no figure is claimed	Not done
Fused versus single-source comparison over repeated seeds	Not done	Not done

4.6.4 Discussion

Three observations follow from the evidence. First, combining evidence changes the outcome in the way the project expected: a single source never passed MEDIUM in our scenario, while the combined incident reached CRITICAL. Second, the physical model on its own is weak on slow change and noisy for operations, so it must not be the only alarm source. Third, the transparent formula made every result easy to check by hand, as the arithmetic in Table 4.4 shows. The limits are those listed in Section 1.5.

4.7 Challenges Encountered and Solutions Adopted

Table 4.7 Challenges and adopted solutions

Challenge	Solution adopted
An early accuracy figure was too optimistic because the same generator produced training and test data	The model is trained on normal simulator runs only and evaluated on separate scenario data, and the older figure is kept as a historical record
Different parts used different speed scales	All detector inputs use the simulator’s RPM scale (0 to 1500) and the documentation states this
The detector and the network layer used different asset names (`M-01`, `MOTOR-01`, `10.10.10.5`)	A configured mapping to the shared asset `PLC-01` in the P2 and P5 configurations
Old or replayed events could raise false warnings and arm the stop timer	Only fresh incidents and live limit alerts can raise warnings or arm the timer; history stays available
A one-shot detection score is not a safe decision	The score is documented as an anomaly indicator, and correlation and rules are applied beside it
Event submission could be abused by any local program	An ingest token is required, and publishers are loopback-only
Simulated response must never act without a human or a clear rule	Confirmation token, loopback checks, countdown, cancellation on normal readings
Audit records could be altered silently	Additional SHA-256 hash chain with a verification page
Many parts, many ports, easy to misstart	A launcher that checks ports and starts services in order

4.8 Chapter Summary

The chapter described the implemented parts and marked which are complete, partial or missing. The automated test suite of 351 tests passed in our verification run. The physical model gave a 4.71% false-positive rate and uneven detection across scenarios, and a fixture-based chain run showed that a correlated network and physical event reached CRITICAL while each alone reached MEDIUM. Live scenario evidence and the fused experiment remain to be added.

Chapter Five: Conclusion and Future Work

5.1 Introduction

This chapter reviews the project against its objectives, summarises the achievements, states the limitations and recommends future work.

5.2 Conclusions

5.2.1 General Objective

The project set out to build a lightweight, AI-assisted early-warning framework for a simulated OT laboratory. A virtual PLC and process, two anomaly models, a rule-based monitor, a correlation and risk engine, and several operator and supervisory interfaces were built and connected. The framework runs on one computer and stays inside a loopback-only, simulation-only boundary. The general objective was met for the prototype. Its detection quality is limited, as Chapter Four showed.

5.2.2 Specific Objectives

Table 5.1 Evaluation of outcomes against objectives

	Specific objective	Outcome
1	Simulated OT environment	Delivered. Virtual PLC with 13 registers, process simulator, HMI, Attack Console
2	Normalised network evidence and scoring	Partly delivered. Adapters and a scored network model work on replayed fixtures, and a live write monitor works on the virtual PLC. Live Zeek and Suricata capture was not implemented
3	Normal telemetry and Isolation Forest baseline	Delivered. 6,000 simulator rows, 11 features. Quality is limited
4	One normalised event format	Delivered
5	Rule-based correlation	Delivered (same asset, 10 seconds)
6	Transparent risk scoring	Delivered. Sensitivity analysis not done
7	Dashboard and Admin console	Delivered. HMI, Admin console, Command Center, Defense Dashboard
8	Evaluation with standard metrics	Done for the physical model. The fused versus single-source comparison over repeated runs was not done
9	Safe, loopback-only scope	Delivered. Enforced in code and tested

5.2.3 Overall Conclusion

Within its limits, the project shows that evidence from the network and from the physical process can be combined by simple rules into one ranked, explained incident, and that this ranking behaves as expected on the fixture scenario. It also shows plainly where the physical detector fails, which is on slow change and with a high false-alarm rate. The next research step is to measure the fused result over many runs.

5.3 Achievements and Contributions

1. A reproducible, loopback-only OT laboratory with a Modbus PLC, a process model and an HMI.
2. A shared event format and a staged pipeline with one JSON hand-off per stage.

3. A network anomaly model, a passive Modbus write monitor and a physical Isolation Forest, each documented with its limits.
4. A transparent risk method and a persistent incident register.
5. A role-based Admin console, a hash-chained audit trail, guided scenarios and printable reports.
6. A separate OT Defense Dashboard with zone policy, safety-setting checks and approval-gated responses.
7. A test suite of 351 automated tests, and an honest evaluation that records negative results.

5.4 Limitations

- **Network evidence is replayed.** No live packet capture. The live monitor covers only Modbus writes to the virtual PLC.
- **Synthetic data only.** Results do not describe real equipment.
- **Detector quality.** High false-alarm rate (339 per hour in the one-hour normal run); weak on slow drift and gradual deviation; `sudden_change` samples were not flagged.
- **Correlation.** One asset and one fixed 10-second window.
- **Risk model.** Weights and thresholds are untested design choices, with no correlation-strength term.
- **Evaluation.** The fused comparison, repeated seeds, load and latency were not measured.
- **Response.** The auto-response module with recovery is not connected to the HMI. The automatic stop acts only on the simulator.
- **Platform.** The Flask applications use development-style local security suitable for a laboratory, not production deployment.
- **Notification.** Telegram is optional and e-mail is not implemented.

5.5 Recommendations and Future Work

5.5.1 Near Term

1. Re-train the physical model on several live simulator runs and tune the contamination setting and threshold on a separate validation set to bring the false-alarm rate down.
2. Add drift-aware features (rolling mean, spread and slope) and a simple drift detector beside the Isolation Forest.
3. Check why `sudden_change` is not flagged and align the scenario generator with the simulator.
4. Run the full comparison of network-only, physical-only and fused detection over several seeds, with precision, recall, F1, false-positive rate and detection latency at the incident level.
5. Add the screenshots and live scenario evidence listed in Table 4.6.

5.5.2 Medium Term

1. Replace replayed fixtures with a live passive monitor of the loopback Modbus traffic, using Zeek or Suricata if resources allow.
2. Test a bounded correlation-strength term and a sensitivity study of the weights, thresholds and the 10-second window.
3. Show, for each alert, which features moved most and the arithmetic behind the score.

5.5.3 Long Term

1. Compare Isolation Forest with other models such as One-Class SVM or Local Outlier Factor.
2. Support several assets and multi-stage incidents.
3. Connect the auto-response module to the HMI only if a human-confirmation design is kept.

5.5.4 Suggested Roadmap

Table 5.2 Suggested roadmap

Horizon	Priority work
Near term	Retrain and tune the physical model, drift features, fused comparison, live evidence
Medium term	Live passive monitoring, correlation-strength term, sensitivity study, explanations
Long term	Model comparison, multi-asset correlation, reviewed response design

5.6 Final Remarks

The project started from one question: whether a network event and an unusual physical state say more together than apart. The prototype gives a working way to ask that question in a safe setting, and the first measurements show both a useful result for correlated evidence and clear weaknesses in the physical detector. Reporting both is what makes the work useful as a base for further study.

References

[1] K. Stouffer, M. Pease, C. Tang, T. Zimmerman, V. Pillitteri, S. Lightman, A. Hahn, S. Saravia, A. Sherule, and M. Thompson, “Guide to Operational Technology (OT) Security,” NIST Special Publication 800-82 Rev. 3, National Institute of Standards and Technology, Sep. 2023.

[2] International Electrotechnical Commission, “Security for industrial automation and control systems,” IEC 62443 series.

[3] Modbus Organization, “MODBUS Application Protocol Specification V1.1b3,” Apr. 2012.

[4] R. Langner, “Stuxnet: Dissecting a cyberwarfare weapon,” *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011.

[5] D. U. Case, “Analysis of the cyber attack on the Ukrainian power grid,” Electricity Information Sharing and Analysis Center (E-ISAC), Mar. 2016.

[6] The Zeek Project, “Zeek documentation.” [Online]. Available: https://docs.zeek.org

[7] Open Information Security Foundation, “Suricata user guide.” [Online]. Available: https://docs.suricata.io

[8] Pallets Projects, “Flask documentation.” [Online]. Available: https://flask.palletsprojects.com

[9] pymodbus Developers, “pymodbus documentation.” [Online]. Available: https://pymodbus.readthedocs.io

[

Appendix A: Repository Layout

Folder	Contents
`chem_shield_p1/`	Virtual PLC, process simulator, HMI, telemetry store
`chem_shield_p2/`	Network adapters, network model, Modbus write monitor, replay tool, fixtures
`chem_shield_p3/`	Physical Isolation Forest, data generators, evaluation scripts, results, graphs
`chem_shield_p4/`	Correlation engine, risk engine, incident store
`chem_shield_p5/`	OT monitor: sensor validation, rules, model scoring
`chem_shield_p6/`	Warning pop-up, simulated response, auto-response module
`chem_shield_p7/`	Command Center, scenarios, audit chain, reports, notifier
`chem_shield_p8/`	OT Defense Dashboard and reactor twin
`chem_shield_common/`	Shared loopback publisher
`admin_part/`	Admin console, user management, audit log, login detector
`the_attack/`	Attack Console
`scripts/`	Launchers, staged chain runner, demo runner, tamper demo

Appendix B: Installation and Execution Guide

One-time setup (from the repository root, Python 3.10 or newer recommended):

1. Create a Python virtual environment:

```
python3 -m venv chem_shield_p1/.venv
```

2. Install the required project dependencies:

```
chem_shield_p1/.venv/bin/python -m pip install -r requirements.txt
```

3. Create an administrator account with the administrator role:

```
chem_shield_p1/.venv/bin/python -m admin_part.manage_users create admin --role administrator
```

The last command asks for a password. On Windows, use

bash

`chem_shield_p1\.venv\Scripts\python.exe` in place of `chem_shield_p1/.venv/bin/python`. The one-command setup is `python scripts/chemshield.py setup`.

Start the machine:

bash

```
chem_shield_p1/.venv/bin/python -m scripts.chemshield run
```

This starts the PLC (15020), process simulator, HMI (15000), Admin console (5000) and Attack Console (5001). Press Ctrl+C once to stop them.

Optional Part 8 dashboard (second terminal):

bash

```
chem_shield_p1/.venv/bin/python -m chem_shield_p8.run
```

Other commands:

bash

```
chem_shield_p1/.venv/bin/python -m scripts.chemshield test      # all tests
chem_shield_p1/.venv/bin/python -m scripts.chemshield status    # port status
chem_shield_p1/.venv/bin/python -m scripts.chemshield stop      # stop services
chem_shield_p1/.venv/bin/python -m scripts.run_chain --step-by-step
chem_shield_p1/.venv/bin/python -m chem_shield_p3.scripts.evaluate_heldout
chem_shield_p1/.venv/bin/python -m scripts.run_pipeline_demo --start-hmi
```

Troubleshooting. If a port is busy, the launcher names the process, and `--kill-old` stops only old CHEM-SHIELD processes. On macOS, port 5000 may be used by AirPlay Receiver, so use `--admin-port 5002`. If the HMI shows the PLC offline, start the PLC first and check port 15020.

Appendix C: User Guide and Demonstration Procedure

1. Start the services as in Appendix B.
2. Open the HMI at http://127.0.0.1:15000 and press START. Use APPLY TARGET to set a speed. Readings should stay below the limits and no incident should appear.
3. Open the Admin console at http://127.0.0.1:5000 and sign in. Review the dashboard, Machine, Incidents and Audit pages.
4. Open the Attack Console at http://127.0.0.1:5001. Enter a value above a limit (for example 92 °C for temperature). The HMI badge turns CRITICAL and the incident appears in the Admin console.
5. Open `/p7/command-center` and run a scenario such as Overheat or Cascade. Watch the countdown and timeline, then generate a report.
6. Open `/p7/integrity` and verify the audit chain.
7. To test the automatic stop quickly, set `CHEM_SHIELD_AUTO_STOP_SECONDS=20` before starting the services.
8. Open http://127.0.0.1:15080 to use the Defense Dashboard, play a scenario and generate its report.

Appendix D: Configuration Details

Setting	Meaning
`CHEM_SHIELD_INGEST_TOKEN`	Token required to submit risk events; random if unset
`CHEM_SHIELD_SECRET_KEY`	Admin session key; random if unset
`CHEM_SHIELD_AUTO_STOP_SECONDS`	Countdown for the simulated stop (default 900)
`CHEM_SHIELD_ADMIN_PORT`	Admin console port (default 5000)
`CHEM_SHIELD_P8_PORT`	Defense Dashboard port (default 15080)
`CHEM_SHIELD_HMI_URL`, `CHEM_SHIELD_ADMIN_URL`	Link targets (HTTP(S) only)
`CHEM_SHIELD_TELEGRAM_BOT_TOKEN`, `CHEM_SHIELD_TELEGRAM_CHAT_ID`	Optional Telegram notifier; unset means no network calls
`chem_shield_p5/ot_monitor/config.example.json`	Limits, rules and model threshold of the OT monitor
`chem_shield_p2/config.example.json`	Asset mapping to `PLC-01`
`chem_shield_p8/config/*.yaml`	Plant, physics, monitor, zones and playbooks

Appendix E: Automated Test Inventory

The 44 test files and their counts are listed in the project; the main ones are:

`admin_part/tests/test_admin.py` (44),

`chem_shield_p8/tests/test_dashboard.py` (26),

`chem_shield_p1/hmi/tests/test_risk_routes.py` (20),

`chem_shield_p5/ot_monitor/tests/test_ot_monitor.py` (18),
`the_attack/tests/test_console.py` (24),

`chem_shield_p6/tests/test_auto_response.py` (14),

`chem_shield_p8/tests/test_plant_twin.py` (15),

`chem_shield_p8/tests/test_ot_monitor.py` (12)

and `chem_shield_p8/tests/test_physics_guard.py` (11).